

LLAMA SCOPE 2: SPARSE DICTIONARIES, REPLACEMENT MODELS, AND THE LLAMASCOPIUM FRAMEWORK

Llama Scope Team
OpenMOSS
TODO: contact email

ABSTRACT

Sparse autoencoders (SAEs) and their variants provide sparse dictionaries for interpreting language model activations in mechanistic interpretability. Furthermore, Transcoders and Low-Rank Sparse Attention (Lorsa) modules provide methods for decomposing MLP and attention computations into interpretable sparse dictionaries. We introduce Llama Scope 2, a unified suite of per-layer Transcoders and Loras modules for the Qwen3, Llama 3.2, and Gemma 3 model families. This release fills a practical gap in current circuit-analysis infrastructure, where popular open language models lack complete and consistent sparse dictionaries for both MLP and attention computations. We evaluate these dictionaries in terms of reconstruction quality, faithfulness, generalization, and scalability, and use case studies to show how they can be composed into replacement models for complete circuit analysis. The checkpoints for the Llama Scope 2 Sparse Dictionary Families are publicly available at [TODO: checkpoint URL], and the training, analysis, and visualization framework is released at [TODO: code URL]. Together, these resources provide an open foundation for broad circuit-level analysis and model biology studies of modern language models.

1 INTRODUCTION

Sparse autoencoders (SAEs) learn sparse and overcomplete dictionaries in which a model activation is represented by a small number of active features (Bricken et al., 2023; Cunningham et al., 2023). These features provide a potentially interpretable decomposition of the information represented within language models and have become foundational tools for research on language model interpretability (He et al., 2024; Lieberum et al., 2024; Deng et al., 2026).

Furthermore, Transcoders decompose MLP computations, enabling preliminary circuit analysis through MLP layers (Dunefsky et al., 2024). Low-Rank Sparse Attention (Lorsa) modules extend this sparse decomposition to attention modules, providing interpretable sparse dictionaries for token-level interactions within attention (He et al., 2025). Together, these dictionaries enable a sparse decomposition, and support circuit analysis across complete Transformers and provide an interpretable view of token-level feature interactions (Shu et al., 2026).

The original Llama Scope release scaled this approach to every layer and sublayer of Llama-3.1-8B and provided a large collection of ready-to-use TopK SAEs (He et al., 2024). This reduced the cost of retraining dictionaries and made it easier to compare sparse features across layers, sites, and dictionary widths.

Llama Scope 2 brings these components together in a unified release spanning Qwen3, Llama 3.2, and Gemma 3 (Deng et al., 2026; McDougall et al., 2025). We use TopK training for the Qwen3 and Llama 3.2 families, and JumpReLU training for Gemma 3 Loras following the recipe adopted by Gemma Scope 2 (McDougall et al., 2025). The release is accompanied by the new version of Llamascopium, a distributed framework that unifies the complete sparse interpretability workflow, including activation generation, sparse dictionary training, feature analysis, visualization, and circuit analysis.

To make circuit analysis more flexible and customizable, we further train Matryoshka SAEs (Bussmann et al., 2025) on the last residual stream in the circuit-tracing pipeline. The resulting hierarchical features allow users to select a residual-feature range of interest while suppressing irrelevant graph components, providing a way to address *Uninterested Circuits* (Shu et al., 2026).

In section 2, we introduce the components of Llama Scope 2: Transcoders, and Low-Rank Sparse Attention (Lorsa). In section 3, we introduce the training procedure for the sparse dictionaries. In section 4, we evaluate the performance of the sparse dictionaries. In section 5, we construct the attribution graph for the sparse dictionaries. In section 6, we demonstrate the use of the sparse dictionaries for case studies. In section 7, we evaluate the quality of the attribution graph.

Contributions. This technical report documents the following components of Llama Scope 2:

- **A multi-family sparse dictionary suite.** We train layer-wise transcoders and Lorsas for Llama 3.2 1B and 3B, Qwen3 1.7B and 8B, and Gemma 3 270M, 1B, and 4B.
- **Replacement models for circuit analysis.** These components can be composed into sparse replacement models that expose feature-to-feature computational pathways while retaining explicit reconstruction-error terms.
- **Matryoshka SAE for uninterested circuits.** We train Matryoshka SAEs on the last residual stream to define hierarchical residual features, enabling *Uninterested Circuits* that prune circuit graphs by selecting a desired feature range.
- **The Llamascopium framework.** We provide a unified, distributed codebase for activation generation, sparse-dictionary training, feature analysis, visualization, and circuit attribution.

2 SPARSE DICTIONARY BACKGROUND

2.1 VANILLA SPARSE AUTOENCODERS

Given a model activation $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$, a sparse autoencoder encodes and reconstructs it as

$$\begin{aligned} \mathbf{h} &= \mathbf{W}_E \mathbf{x} + \mathbf{b}_E, & \mathbf{z} &= \sigma(\mathbf{h}), \\ \hat{\mathbf{x}} &= \mathbf{W}_D \mathbf{z} + \mathbf{b}_D. \end{aligned} \quad (1)$$

Here, $\mathbf{h}$ denotes the encoder pre-activations and $\mathbf{z}$ the sparse feature activations. The columns of $\mathbf{W}_D$ form an overcomplete dictionary of feature directions, while the nonzero entries of $\mathbf{z}$ specify how these directions are linearly combined to reconstruct $\mathbf{x}$ (up to the decoder bias $\mathbf{b}_D$). Llama Scope 2 primarily uses two activation functions: TopK, which directly controls the number of active latents per token, and JumpReLU, which learns a separate activation threshold for each latent (Gao et al., 2024; Rajamanoharan et al., 2024).

2.2 PER-LAYER TRANSCODERS

Per-layer transcoders have the same encoder–decoder structure as sparse autoencoders, but differ in their training objective (Dunefsky et al., 2024). Rather than reconstructing a representation from itself, a transcoder takes the normalized input to an MLP sublayer and predicts the corresponding MLP output. In this sense, transcoders provide a sparse decomposition of the MLP computation, rather than of a single representation.

For each layer ℓ , the transcoder maps the normalized MLP input $\mathbf{x}_{\text{MLP},\text{in}}$ to the corresponding MLP output $\mathbf{x}_{\text{MLP},\text{out}}$ as

$$\begin{aligned} \mathbf{h} &= \mathbf{W}_E \mathbf{x}_{\text{MLP},\text{in}} + \mathbf{b}_E, & \mathbf{z} &= \sigma(\mathbf{h}), \\ \hat{\mathbf{x}}_{\text{MLP},\text{out}} &= \mathbf{W}_D \mathbf{z} + \mathbf{b}_D. \end{aligned} \quad (2)$$

2.3 LOW-RANK SPARSE ATTENTION (LORSA)

Lorsa can be viewed as a transcoder-style decomposition of attention layers (He et al., 2025). It represents attention as a sparse dictionary of rank-1 features, where the OV weights serve as

the decoder and decoder, while the QK pattern determines feature activation and represents the interactions across token positions.

For an attention input $\mathbf{x}_{\text{Attn,in}} \in \mathbb{R}^{n \times d_{\text{model}}}$, each Lorsa feature k first computes its attention pattern as

$$\begin{aligned} \mathbf{A}_k &= \text{softmax}\left(\frac{(\mathbf{x}_{\text{Attn,in}} \mathbf{W}_{Q,k})(\mathbf{x}_{\text{Attn,in}} \mathbf{W}_{K,k})^\top}{\sqrt{d_h}} + \mathbf{B}\right), \\ h_{i,k} &= \sum_{j=1}^n A_{k,i,j} \mathbf{w}_{V,k}^\top \mathbf{x}_{\text{Attn,in},j}. \end{aligned} \quad (3)$$

The sparse feature activation is obtained by applying the activation function

$$z_{i,k} = \sigma(h_{i,k}). \quad (4)$$

Finally, the active features are decoded into the attention output as

$$\hat{\mathbf{x}}_{\text{Attn,out},i} = \sum_k z_{i,k} \mathbf{w}_{O,k} + \mathbf{b}_O. \quad (5)$$

Here, $z_i \in \mathbb{R}^{n \times 1}$ denotes the scalar activation of feature i across token positions, and $\mathbf{w}_{O,i}$ specifies its output direction. In this sense, Lorsa provides a sparse decomposition of the attention computation.

2.4 MATRYOSHKA SPARSE AUTOENCODERS

Matryoshka Sparse Autoencoders (Matryoshka SAEs) extend standard SAEs by organizing the latent space into a sequence of nested prefixes (Bussmann et al., 2025). Each prefix is trained to reconstruct the same activation, encouraging earlier features to capture more general structure and later features to provide progressively finer-grained representations.

Given an activation $\mathbf{x}$, the encoder is the same as in a standard SAE,

$$\mathbf{h} = \mathbf{W}_E \mathbf{x} + \mathbf{b}_E, \quad \mathbf{z} = \sigma(\mathbf{h}), \quad (6)$$

while different prefixes $\mathbf{z}^{(m)}$ of the latent vector are decoded independently,

$$\hat{\mathbf{x}}^{(m)} = \mathbf{W}_D \mathbf{z}^{(m)} + \mathbf{b}_D, \quad m \in \mathcal{M}, \quad (7)$$

where $\mathcal{M}$ denotes the set of predefined prefix widths used to form the nested latent representations. We train Matryoshka SAEs on the final residual stream of the model, immediately before the unembedding layer. We find that they can organize concepts into hierarchical levels, for example, from “person name” to “female name” and finally to a specific name such as “Mary.” This hierarchy allows circuit tracing to selectively identify circuits at different levels of abstraction.

3 TRAINING LLAMA SCOPE 2

3.1 RELEASE OVERVIEW

Table ?? summarizes the planned organization of the release. The table records the configurations currently represented by the main training workflows. The final public-checkpoint count and any additional SAE or Matryoshka variants will be added after the final checkpoint audit.

All Lorsa modules with `initialize_lorsa_with_mhsa` enabled initialize their QK projections from the corresponding pretrained attention layer. For GQA backbones, the key projections are expanded from key-value heads to query-head slots before this copy, so repeated Lorsa K heads share the same initial value whenever the base model shares a key head. These heads are not tied during optimization; the shared-K structure is an initialization prior rather than a hard parameter-sharing constraint.

Table 1: Core Llama Scope 2 training configurations. TopK values are per token. For JumpReLU Lorsas, L_0^* is layer-dependent; arrows denote the target- L_0 schedules from early to deeper layers. Each listed configuration is trained layer-wise.

Backbone	Layers	d_{model}	Dictionary	Width	Activation / sparsity	Training tokens
Llama 3.2 1B	16	2048	TC, Lorsa	$8\times, 32\times$	TopK, $K \in \{64, 128, 256\}$	8B
Llama 3.2 3B	28	3072	TC, Lorsa	$8\times, 32\times$	TopK, $K \in \{64, 128, 256\}$	8B
Qwen3 1.7B	28	2048	TC, Lorsa	$8\times, 32\times$	TopK, $K \in \{64, 128, 256\}$	8B
Qwen3 8B	36	4096	TC, Lorsa	$8\times, 32\times$	TopK, $K = 256$	8B
Gemma 3 270M	18	640	Lorsa	16,384	JReLU, $L_0^* : 10 \rightarrow 20, 60 \rightarrow 120$	8B
Gemma 3 1B	26	1152	Lorsa	16,384	JReLU, $L_0^* : 10 \rightarrow 20, 60 \rightarrow 120$	8B
Gemma 3 4B	34	2560	Lorsa	16,384	JReLU, $L_0^* : 10 \rightarrow 20, 60 \rightarrow 120$	8B

Table 2: Lorsa attention configurations used in our workflow scripts. `n_qk_heads` is the number of Lorsa query-key heads, and H_{OV} is the number of sparse output-value heads. Braced pairs list the values for $8\times$ and $32\times$ dictionaries.

Backbone	Width	<code>n_qk_heads</code>	H_{OV}	d_{QK}	RoPE	Attention	Post-QK norm
Llama 3.2 1B	$8\times, 32\times$	256, 1024	16,384, 65,536	64	$\theta = 5\times 10^5$, NTK-by-parts	global	none
Llama 3.2 3B	$8\times, 32\times$	192, 768	24,576, 98,304	128	$\theta = 5\times 10^5$, NTK-by-parts	global	none
Qwen3 1.7B	$8\times, 32\times$	128, 512	16,384, 65,536	128	$\theta = 10^6$	global	RMS
Qwen3 8B	$8\times, 32\times$	256, 1024	32,768, 131,072	128	$\theta = 10^6$	global	RMS
Gemma 3 270M	16,384	64	16,384	256	$\theta = 10^6 / \text{local } 10^4$	local/global, $w = 512$	RMS
Gemma 3 1B	16,384	64	16,384	256	$\theta = 10^6 / \text{local } 10^4$	local/global, $w = 512$	RMS
Gemma 3 4B	16,384	64	16,384	256	$\theta = 10^6 / \text{local } 10^4$	local/global, $w = 1024$	RMS

3.2 MODEL OVERVIEW

3.2.1 QWEN3 MODELS

We train per-layer Transcoders and Lorsas for Qwen3-1.7B and Qwen3-8B (Yang et al., 2025). Both Qwen3 variants use global causal grouped-query attention (GQA) (Ainslie et al., 2023) with rotary position embeddings and per-head QK RMS normalization. We match these details in Lorsa by using the native QK geometry: Qwen3-1.7B has 16 query heads and Qwen3-8B has 32 query heads, both with 8 key-value heads, 128-dimensional QK heads, RoPE base 10^6 , and post-QK RMS normalization. The number of Lorsa QK heads is scaled by the dictionary expansion factor. Both releases use TopK activation with $8\times$ and $32\times$ dictionary expansion factors. The 1.7B suite includes $K \in \{64, 128, 256\}$, while the 8B suite uses $K = 256$.

3.2.2 LLAMA 3.2 MODELS

For Llama 3.2 1B and 3B (Grattafiori et al., 2024), we train per-layer Transcoders and Lorsas across all Transformer layers. Llama 3.2 uses global causal GQA with rotary position embeddings and Llama-3 long-context rotary scaling, but does not use QK normalization or local sliding-window attention. We therefore instantiate Lorsa with the native query-head counts and head dimensions—32 heads of dimension 64 for the 1B model and 24 heads of dimension 128 for the 3B model, with 8 key-value heads in both base models—and disable post-QK normalization. Both model sizes use $8\times$ and $32\times$ dictionaries with $K \in \{64, 128, 256\}$.

3.2.3 GEMMA 3 MODELS

For Gemma 3 270M, 1B, and 4B, we train per-layer Lorsa modules with 16,384 sparse OV features. We use JumpReLU activations and layer-dependent sparsity targets following Gemma Scope 2 (McDougall et al., 2025).

Because Gemma uses QK normalization, grouped-query attention, and interleaved local/global attention (Team et al., 2025; Ainslie et al., 2023), we adapt Lorsa to match these architectural details, including the corresponding QK head structure, local attention configuration, and QK normalization. We initialize the Lorsa QK parameters from the pretrained Gemma attention weights described above (Wang et al., 2026), while sharing key projections according to Gemma’s GQA structure.

Architecture adaptation for Gemma. Gemma attention differs from standard multi-head self-attention in several important ways: it uses QK normalization, grouped-query attention (GQA), and an interleaved pattern of local sliding-window and global attention layers, with different rotary settings across these attention types. We adapt Lorsa to match these architectural details (Team et al., 2025; Ainslie et al., 2023). In particular, we apply Gemma’s QK normalization inside the Lorsa attention computation, use the appropriate local window and rotary configuration for each layer, and align the QK head structure with Gemma’s GQA layout while keeping 16,384 sparse OV features. The pretrained-attention initialization gives the sparse decomposition an architecture-matched starting point (Wang et al., 2026).

3.3 ACTIVATION DATA

For sweeping, training, and analysis (visualization), we use datasets drawn from the same distribution to generate activations at the corresponding hook points. Llama 3.2 and the Gemma 3 models use 2,048-token contexts from FineWeb-Edu (Penedo et al., 2024). Qwen3 uses 2,048-token contexts from an internal uncropped code–English–Chinese pretraining corpus. During training, we apply dataset-wise activation normalization using a fixed scale estimated from the training distribution. This brings activations to a stable average norm and improves the numerical scale of parameter optimization, while preserving relative token-level magnitudes. After training, we fold the normalization scale into the learned parameters, allowing the sparse dictionaries to operate directly on activations in their original scale at inference time. Cached activations are stored in bfloat16 precision to reduce storage and memory overhead. During training, we use mixed precision with bfloat16 autocast for the forward and loss computation, while sparse-dictionary parameters are stored in float32 unless otherwise specified.

3.4 TRAINING DETAILS

For the Qwen3 and Llama 3.2 model families, we train TopK Transcoders and Lorsas. For the Gemma 3 family, Gemma Scope 2 already provides JumpReLU SAEs and Transcoders; we therefore train only the complementary Lorsas. Below we describe the training objectives, initialization and optimization details for the TopK Transcoders and Lorsas.

3.4.1 TOPK TRANSCODERS AND LORSAS

Objectives For TopK Transcoders and Lorsas, sparsity is enforced directly by retaining the K largest positive pre-activations:

$$\mathbf{z} = \text{TopK}(\text{ReLU}(\mathbf{h}), K). \quad (8)$$

The model is therefore trained using only the reconstruction objective,

$$\mathcal{L} = \mathcal{L}_{\text{rec}} = \mathbb{E} \left[\|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 \right], \quad (9)$$

without additional sparsity penalties.

Initialization For TopK Transcoders and Lorsas, we initialize the decoder and encoder weights from uniform distributions with scale $1/\sqrt{d_{\text{model}}}$ and $1/\sqrt{d_{\text{model}}m}$, respectively, where m is the dictionary expansion factor. We do not initialize the encoder as the decoder transpose. Instead, after sampling the weights, we perform a coarse-to-fine grid search over the decoder norm on an activation batch and choose the norm that minimizes the initial reconstruction loss (Gao et al., 2024). We initialize the decoder bias from the empirical center of the normalized target activations and initialize the encoder bias from the mean pre-activation. For Transcoders, we additionally initialize from the corresponding pretrained MLP layer. For Lorsas, we initialize the QK parameters from the corresponding pretrained attention layer and initialize the OV decoder directions within the active subspace of the training activations (Wang et al., 2026).

Optimization For all TopK Transcoders and Lorsas, the initial TopK value is equal to the final target K . Transcoders use SparseAdam, while Lorsas use Adam; both use $(\beta_1, \beta_2) = (0.9, 0.999)$ and no weight decay. We use 5,000 warmup steps followed by a linear cooldown over the final 20% of training, with no gradient clipping.

TopK Transcoders use a batch size of 32,768 tokens. TopK Lorsa batch sizes are 32,768 for Qwen3 and $8 \times$ Llama, and 16,384 for $32 \times$ Llama. Learning rates and other hyperparameters are summarized in Appendix B.1.

3.4.2 JUMPReLU LORSA

Gemma Scope 2 provides a comprehensive suite of JumpReLU SAEs and Transcoders for Gemma 3. Following this design, we use JumpReLU activations when training Loras for the Gemma 3 models.

Objectives. The reconstruction objective is

$$\mathcal{L}_{\text{rec}} = \mathbb{E} \left[\| \mathbf{x} - \hat{\mathbf{x}} \|_2^2 \right]. \quad (10)$$

We encourage a target sparsity level L_0^* using

$$\mathcal{L}_{\text{sparse}} = \lambda \frac{2}{L_0^*} (L_0 - L_0^*)^2, \quad (11)$$

where λ is the sparsity coefficient (McDougall et al., 2025).

To mitigate dead features, we additionally use an auxiliary loss (Rajamanoharan et al., 2024),

$$\mathcal{L}_{\text{aux}} = \lambda_{\text{aux}} \mathbb{E} \left[\sum_{i \in \mathcal{D}} \max(0, \tau_i - \tilde{z}_i) \right], \quad (12)$$

where $\mathcal{D}$ denotes the set of dead latents, τ_i is the detached JumpReLU threshold, $\tilde{z}_i$ is the corresponding pre-activation, and λ_{aux} controls the auxiliary loss strength.

The final training objective is

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{sparse}} + \mathcal{L}_{\text{aux}}. \quad (13)$$

Initialization. Gemma Loras are initialized from the original multi-head self attention layers. We copy the query projections directly and initialize the key projections according to Gemma’s GQA structure, so heads sharing a pretrained key-value head start from the same key projection. The repeated key heads are then trained independently. We also initialize the OV decoder directions from the active subspace of the training attention outputs as describe in TopK Lorsa.

Optimization. Gemma Loras are trained with Adam. We use 5,000 warmup steps and linearly decay the learning rate over the final 20% of training to $1/32$ of its peak value. To avoid the excessive number of dead features, we linearly warm up the ℓ_1 coefficient, and use an auxiliary loss to further improve dictionary utilization.

For the JumpReLU threshold, we use a separate learning rate for the learnable threshold parameters. We find that unconstrained threshold optimization can cause both the JumpReLU thresholds and encoder norms to grow excessively. To prevent this, we freeze the JumpReLU thresholds once the L_0 approaches the target L_0 . We additionally apply global gradient clipping at 1.0 to stabilize threshold optimization and set the L_p coefficient to 0.003.

The token batch sizes are 16,384, 32,768, and 131,072 for Gemma 3 270M, 1B, and 4B, respectively. Learning rates and other hyperparameters are summarized in Appendix B.2.

4 SPARSE DECOMPOSITION EVALUATION

4.1 EVALUATION SETUP AND METRICS

We evaluate the learned sparse decompositions along four dimensions: dictionary quality, scaling behavior, model-level fidelity, and feature interpretability.

Dictionary quality. We report explained variance (EV), L_0 , latent density, firing frequency, and the number of dead latents to characterize reconstruction quality and feature utilization.

Scaling behavior. We evaluate how reconstruction quality changes as the dictionary width and TopK sparsity level K increase, focusing primarily on the scaling of EV.

Model-level fidelity. We insert the trained Transcoders and Lorsas into the model and compare the replacement model against the original model using output KL divergence and the change in language-modeling loss, $\Delta\mathcal{L}_{LM}$.

Feature interpretability. We automatically generate and evaluate feature explanations following prior automated interpretability protocols (Cunningham & Conerly, 2024; Karvonen et al., 2025), and report the corresponding automated interpretability scores.

Feature steering. We evaluate the controllability of sparse features through activation steering (Rimsky et al., 2024). Specifically, we consider two steering tasks: (i) text-style transfer, following Qwen-Scope (Deng et al., 2026), and (ii) personality steering, where we amplify personality-related features to induce corresponding behavioral traits (Chen et al., 2025).

4.2 SPARSE DICTIONARY QUALITY

We first evaluate the reconstruction–sparsity trade-off of the trained Transcoders and Lorsas across model families and layers. We report EV and L_0 , together with firing-frequency, latent-density, and dead-latent statistics.

4.3 SCALING BEHAVIOR

We next study how sparse decomposition quality scales with dictionary capacity. For TopK models, we vary both the dictionary expansion factor and K and measure the corresponding change in EV. This allows us to examine whether larger dictionaries and denser operating points consistently improve reconstruction across model families.

4.4 REPLACEMENT FIDELITY

Reconstruction quality alone does not guarantee that a sparse module faithfully preserves the original model computation. We therefore replace the original MLP and attention modules with the corresponding Transcoders and Lorsas and measure the resulting change in model behavior. We report output KL divergence and $\Delta\mathcal{L}_{LM}$, with lower values indicating higher replacement fidelity.

4.5 FEATURE INTERPRETABILITY

Finally, we evaluate whether the learned sparse features admit meaningful natural-language descriptions. We apply automated feature interpretation to Transcoder and Lorsa features and evaluate the generated explanations using automated interpretability scores (Cunningham & Conerly, 2024; Karvonen et al., 2025).

5 ATTRIBUTION GRAPH CONSTRUCTION

We use a per-layer Transcoder to sparsify each MLP sublayer and a Lorsa to sparsify each attention sublayer. We additionally place a Matryoshka SAE on the final residual stream as an abstraction-selection interface for graph construction. Together, these modules provide a sparse replacement of the full Transformer computation, over which we construct attribution graphs following circuit tracing (Ameisen et al., 2025; Shu et al., 2026).

Following Shu et al. (2026), we define an attribution graph as

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A}), \quad (14)$$

where the node set consists of

$$\mathcal{V} = \mathcal{V}_{\text{embed}} \cup \mathcal{V}_{\text{transcoder}} \cup \mathcal{V}_{\text{lorsa}} \cup \mathcal{V}_{\text{matryoshka}} \cup \mathcal{V}_{\text{error}} \cup \mathcal{V}_{\text{logit}}. \quad (15)$$

For a source node s and a downstream target node t , their effective encoder and decoder directions depend on the node type:

$$\begin{aligned} \mathbf{w}_{\text{enc},t} &= \begin{cases} \mathbf{w}_{\text{enc},t}, & t \text{ is a Transcoder or Matryoshka feature,} \\ \mathbf{w}_{V,t}, & t \text{ is a Lorsa feature,} \\ \mathbf{w}_{\text{unembed},t}, & t \text{ is a logit,} \end{cases} \\ \mathbf{w}_{\text{dec},s} &= \begin{cases} \mathbf{w}_{\text{dec},s}, & s \text{ is a Transcoder or Matryoshka feature,} \\ \mathbf{w}_{O,s}, & s \text{ is a Lorsa feature,} \\ \mathbf{w}_{\text{embed},s}, & s \text{ is a token embedding,} \\ \epsilon_s, & s \text{ is an error node.} \end{cases} \end{aligned} \quad (16)$$

The direct attribution from s to t is

$$\alpha_{s \rightarrow t} = a_s P_{i,j}^t \mathbf{w}_{\text{dec},s}^\top \mathbf{w}_{\text{enc},t}, \quad (17)$$

where a_s is the source activation and $P_{i,j}^t$ captures the token-to-token interaction associated with the target computation, with $P^t = I$ for non-attention components.

Starting from a target logit, we recursively trace high-attribution edges backward through the sparse replacement model to obtain the attribution graph. In practice, we compute these scores using gradients while detaching attention patterns and LayerNorm denominators, following Ameisen et al. (2025).

5.1 MATRYOSHKA SAEs FOR UNINTERESTED CIRCUITS

Full attribution graphs often contain pathways that are faithful to the target computation but irrelevant to the analyst’s question. We refer to these components as *uninterested circuits*. Llama Scope 2 addresses this problem by using a Matryoshka SAE on the final residual stream to choose the abstraction level before attribution is traced.

Let the Matryoshka widths be $0 = b_0 < b_1 < \dots < b_M = d_{\text{SAE}}$. For a residual activation $\mathbf{x}_i$ at token position i , the Matryoshka SAE produces latent activations $\mathbf{z}_i \in \mathbb{R}^{d_{\text{SAE}}}$. During circuit construction, the user selects a feature range $R = [b_a, b_b)$ whose endpoints are Matryoshka boundaries. We expose only features in this range by applying the mask

$$z_{i,k}^{(R)} = z_{i,k} \mathbf{1}\{k \in R\}, \quad (18)$$

and use the corresponding residual reconstruction

$$\hat{\mathbf{x}}_i^{(R)} = \mathbf{b}_D + \sum_{k \in R} z_{i,k} \mathbf{w}_{D,k}, \quad \epsilon_i^{(R)} = \mathbf{x}_i - \hat{\mathbf{x}}_i^{(R)}. \quad (19)$$

This induces a range-conditioned attribution graph $\mathcal{G}_R = (\mathcal{V}_R, \mathcal{E}_R, \mathcal{A}_R)$ whose Matryoshka nodes are restricted to

$$\mathcal{V}_{\text{matryoshka}}^{(R)} = \{(i, k) : z_{i,k} \neq 0, k \in R\}. \quad (20)$$

Features outside R are not available as graph nodes; their omitted contribution remains in the explicit residual reconstruction-error term $\epsilon_i^{(R)}$. Choosing a prefix $[0, b_j)$ traces the computation at a cumulative abstraction level, while choosing an adjacent segment $[b_{j-1}, b_j)$ isolates the incremental features added at that level. In this way, Matryoshka SAEs turn uninterested circuits into a graph-construction choice rather than a post-hoc pruning problem.

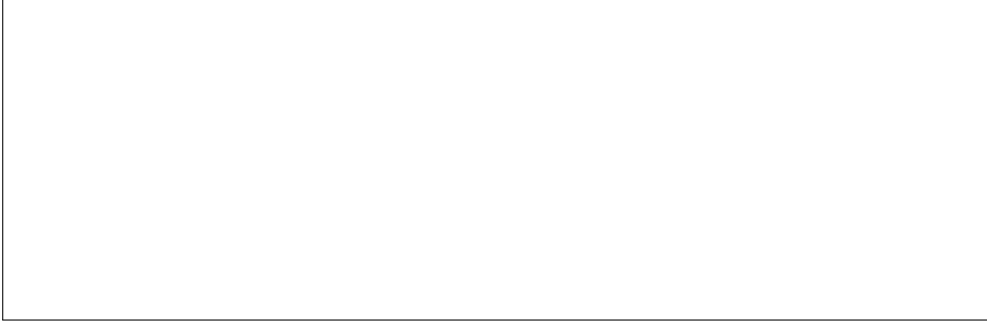


Figure 1: Overview of Llama Scope 2. The final figure will show how sparse autoencoders, transcoders, and Lorsas cover the principal representations and computations in a Transformer block.

6 CASE STUDIES

6.1 FACTUAL RECALL

6.2 SHORTCUT REASONING

6.3 MODEL CIRCUIT COMPARISON

6.4 UNINTERESTED CIRCUIT

7 ATTRIBUTION GRAPH EVALUATION

We evaluate the quality of attribution graphs using two graph-level metrics, the *replacement score* and the *completeness score*, following prior circuit-tracing work (Ameisen et al., 2025; Shu et al., 2026). Both metrics quantify the extent to which the traced computation is explained by sparse features rather than reconstruction errors.

7.1 GRAPH SCORES

Let $\mathcal{P}_{\text{feat}}$, $\mathcal{P}_{\text{error}}$, and $\mathcal{P}_{\text{emb}}$ denote attribution paths originating from sparse feature nodes, reconstruction-error nodes, and input embeddings, respectively, and let $w(p)$ denote the attribution strength of path p .

Replacement score measures the fraction of all edges from the attribution graph that originate from feature nodes rather than error nodes, weighted by their attribution strengths:

$$S_{\text{repl}} = \frac{\sum_{p \in \mathcal{P}_{\text{feat}}} w(p)}{\sum_{p \in \mathcal{P}_{\text{feat}}} w(p) + \sum_{p \in \mathcal{P}_{\text{error}}} w(p)}. \quad (21)$$

Completeness score measures the fraction of total attribution to the logit that can be traced all the way back to the input embeddings, rather than to error nodes:

$$S_{\text{comp}} = \frac{\sum_{p \in \mathcal{P}_{\text{emb}}} w(p)}{\sum_{p \in \mathcal{P}_{\text{emb}}} w(p) + \sum_{p \in \mathcal{P}_{\text{error}}} w(p)} = 1 - \frac{\sum_{p \in \mathcal{P}_{\text{error}}} w(p)}{\sum_{p \in \mathcal{P}_{\text{emb}}} w(p) + \sum_{p \in \mathcal{P}_{\text{error}}} w(p)}. \quad (22)$$

Higher scores indicate that a larger fraction of the traced computation is accounted for by sparse, interpretable replacement features. For complete replacement models, these features include both transcoder features for MLP computation and Lorsa features for attention computation.

7.2 PERTURBATION FIDELITY

We evaluate whether the replacement model preserves the downstream effect of an upstream perturbation. We apply the same perturbation to the original and replacement models and measure the cosine

similarity between their resulting downstream activation changes. A higher similarity indicates that the replacement model better preserves the causal propagation of the original model.

7.3 ATTRIBUTION-INTERVENTION CONSISTENCY

We further test whether attribution scores predict the actual causal effects of feature interventions. We steer individual features and measure their effects on the output logits, then compute the Pearson correlation between these intervention effects and the corresponding attribution scores. A higher correlation indicates that the attribution graph more faithfully reflects the causal influence of its features.

8 THE LLAMASCOPIUM FRAMEWORK

Llamascopium is the framework used to construct, analyze, and serve the Llama Scope 2 release. Its design separates sparse-dictionary architectures from data production, distributed execution, analysis, and visualization, allowing the same pipeline to support multiple model families and dictionary types.

8.1 UNIFIED SPARSE-DICTIONARY INTERFACE

8.2 ACTIVATION PIPELINE

The activation pipeline provides a common interface for generating, loading, filtering, shuffling, and batching model activations. It supports both flattened one-dimensional activations and sequence-preserving two-dimensional activations.

8.3 DISTRIBUTED RUNNERS

Llamascopium exposes high-level runners for activation generation, training, evaluation, and feature analysis. These runners support combinations of data, tensor, model, and head parallelism according to the dictionary architecture.

8.4 ANALYSIS, STORAGE, AND VISUALIZATION

Feature-level results are stored in a database-backed representation and served through an analysis API and web interface. Supported analyses include activation sampling, firing statistics, activation histograms, direct logit attribution, automated interpretation, and architecture-specific metadata such as Lorsa source-target patterns.

8.5 CIRCUIT ANALYSIS ENGINE

The circuit engine composes sparse replacement modules with the language model and traces attribution from output logits through residual, MLP, and attention latents. Reconstruction-error nodes remain explicit in the resulting graph.

9 LIMITATIONS AND OPEN PROBLEMS

Dark matter of sparse decomposition.

Inhibitory Circuits.

10 CONCLUSION

Llama Scope 2 extends sparse dictionary releases from representation-level feature vocabularies toward sparse decompositions of Transformer computation. This report documents a unified suite for



Figure 2: Planned architecture diagram for Llamascopium, covering model backends, activation factories, sparse dictionaries, distributed runners, analysis storage, visualization, and circuit tracing.

Qwen3, Llama 3.2, and Gemma 3 models and Llamascopium, the framework used to train, analyze, visualize, and compose these dictionaries for circuit analysis.

ACKNOWLEDGMENTS

REFERENCES

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: training generalized multi-query transformer models from multi-head checkpoints. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pp. 4895–4901. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.EMNLP-MAIN.298. URL <https://doi.org/10.18653/v1/2023.emnlp-main.298>.
- Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- Bart Bussmann, Noa Nabeshima, Adam Karvonen, and Neel Nanda. Learning multi-level features with matryoshka sparse autoencoders. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, volume 267 of *Proceedings of Machine Learning Research*. PMLR / OpenReview.net, 2025. URL <https://proceedings.mlr.press/v267/bussmann25a.html>.
- Runjin Chen, Andy Ardit, Henry Sleight, Owain Evans, and Jack Lindsey. Persona vectors: Monitoring and controlling character traits in language models. *CoRR*, abs/2507.21509, 2025. doi: 10.48550/ARXIV.2507.21509. URL <https://doi.org/10.48550/arXiv.2507.21509>.

- Hoagy Cunningham and Tom Conerly. Circuits updates - june 2024. Transformer Circuits Thread, 2024. URL <https://transformer-circuits.pub/2024/june-update/index.html>.
- Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *CoRR*, abs/2309.08600, 2023. doi: 10.48550/ARXIV.2309.08600. URL <https://doi.org/10.48550/arXiv.2309.08600>.
- Boyi Deng, Xu Wang, Yaoning Wang, Yu Wan, Yubo Ma, Baosong Yang, Haoran Wei, Jialong Tang, Huan Lin, Ruize Gao, Tianhao Li, Qian Cao, Xuancheng Ren, Xiaodong Deng, An Yang, Fei Huang, Dayiheng Liu, and Jingren Zhou. Qwen-scope: Turning sparse features into development tools for large language models. *CoRR*, abs/2605.11887, 2026. doi: 10.48550/ARXIV.2605.11887. URL <https://doi.org/10.48550/arXiv.2605.11887>.
- Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. Transcoders find interpretable LLM feature circuits. *CoRR*, abs/2406.11944, 2024. doi: 10.48550/ARXIV.2406.11944. URL <https://doi.org/10.48550/arXiv.2406.11944>.
- Leo Gao et al. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Zhengfu He, Wentao Shu, Xuyang Ge, Lingjie Chen, Junxuan Wang, Yunhua Zhou, Frances Liu, Qipeng Guo, Xuanjing Huang, Zuxuan Wu, Yu-Gang Jiang, and Xipeng Qiu. Llama scope: Extracting millions of features from llama-3.1-8b with sparse autoencoders. *arXiv preprint arXiv:2410.20526*, 2024.
- Zhengfu He et al. Towards understanding the nature of attention with low-rank sparse decomposition. *arXiv preprint arXiv:2504.20938*, 2025.
- Adam Karvonen, Can Rager, Johnny Lin, Curt Tigges, Joseph Isaac Bloom, David Chanin, Yeu-Tong Lau, Eoin Farrell, Callum McDougall, Kola Ayonrinde, Demian Till, Matthew Wearden, Arthur Conmy, Samuel Marks, and Neel Nanda. Saebench: A comprehensive benchmark for sparse autoencoders in language model interpretability. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, volume 267 of *Proceedings of Machine Learning Research*. PMLR / OpenReview.net, 2025. URL <https://proceedings.mlr.press/v267/karvonen25a.html>.
- Tom Lieberum, Senthooan Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda. Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2. In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pp. 278–300, 2024.
- Callum McDougall, Arthur Conmy, János Kramár, Tom Lieberum, Senthooan Rajamanoharan, and Neel Nanda. Gemma scope 2: Technical paper. Technical report, Google DeepMind, 2025.
- Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin A. Raffel, Leandro von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (eds.), *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/370df50ccfd8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html.
- Senthooan Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár, and Neel Nanda. Jumping ahead: Improving reconstruction fidelity with jumprelu sparse autoencoders. *CoRR*, abs/2407.14435, 2024. doi: 10.48550/ARXIV.2407.14435. URL <https://doi.org/10.48550/arXiv.2407.14435>.

Nina Rimskey, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering llama 2 via contrastive activation addition. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2024, Bangkok, Thailand, August 11-16, 2024, pp. 15504–15522. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.828. URL <https://doi.org/10.18653/v1/2024.acl-long.828>.

Wentao Shu, Xuyang Ge, Guancheng Zhou, Junxuan Wang, Rui Lin, Zhaoxuan Song, Jiaying Wu, Zhengfu He, and Xipeng Qiu. Bridging the attention gap: Complete replacement models for complete circuit tracing. *OpenMOSS Interpretability Research*, 2026. URL <https://interp.open-moss.com/posts/complete-replacement>.

Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

Junxuan Wang, Xuyang Ge, Wentao Shu, Zhengfu He, and Xipeng Qiu. Dimensional collapse in transformer attention outputs: A challenge for sparse dictionary learning. In *Forty-Third International Conference on Machine Learning, ICML 2026, Seoul, South Korea, July 6–11, 2026*, 2026. URL <https://icml.cc/virtual/2026/poster/64760>.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

A RELEASED CHECKPOINTS

B TRAINING HYPERPARAMETERS

B.1 TOPK TRAINING HYPERPARAMETERS

B.2 GEMMA 3 JUMPReLU LORSA HYPERPARAMETERS

C ADDITIONAL EVALUATION RESULTS

D ANALYSIS AND VISUALIZATION DETAILS

E LLAMASCOPIUM API AND REPRODUCIBILITY

Table 3: Training hyperparameters for TopK Transcoders and Lorsa modules on the Llama 3.2 models.

Model	Width	Module	K	Learning rate	λ_1
Llama 3.2 1B	$8\times$	Transcoder	64	1×10^{-4}	8×10^{-5}
			128	1×10^{-4}	8×10^{-5}
			256	1×10^{-4}	8×10^{-5}
		Lorsa	64	6×10^{-5}	8×10^{-5}
			128	6×10^{-5}	8×10^{-5}
			256	7×10^{-5}	8×10^{-5}
	$32\times$	Transcoder	64	1×10^{-4}	8×10^{-5}
			128	1×10^{-4}	8×10^{-5}
			256	1×10^{-4}	8×10^{-5}
		Lorsa	64	2×10^{-4}	8×10^{-5}
			128	2×10^{-4}	8×10^{-5}
			256	1×10^{-4}	8×10^{-5}
Llama 3.2 3B	$8\times$	Transcoder	64	1×10^{-4}	8×10^{-5}
			128	1×10^{-4}	8×10^{-5}
			256	9×10^{-5}	8×10^{-5}
		Lorsa	64	6×10^{-5}	8×10^{-5}
			128	6×10^{-5}	8×10^{-5}
			256	6×10^{-5}	8×10^{-5}
	$32\times$	Transcoder	64	1×10^{-4}	8×10^{-5}
			128	8×10^{-5}	8×10^{-5}
			256	9×10^{-5}	8×10^{-5}
		Lorsa	64	6×10^{-5}	8×10^{-5}
			128	6×10^{-5}	8×10^{-5}
			256	6×10^{-5}	8×10^{-5}

Table 4: Training hyperparameters for TopK Transcoders and Lorsa modules on the Qwen3 models. For Qwen3 1.7B, $\mathcal{S}_{lr} = \{0.6, 0.7, 0.8, 0.9, 1, 2, 3, 4\} \times 10^{-4}$ denotes the swept learning-rate set.

Model	Width	Module	K	Learning rate	λ_1
Qwen3 1.7B	$8\times$	Transcoder	64	$\mathcal{S}_{lr}$	—
			128	$\mathcal{S}_{lr}$	—
			256	$\mathcal{S}_{lr}$	—
		Lorsa	64	$\mathcal{S}_{lr}$	—
			128	$\mathcal{S}_{lr}$	—
			256	$\mathcal{S}_{lr}$	—
	$32\times$	Transcoder	64	$\mathcal{S}_{lr}$	—
			128	$\mathcal{S}_{lr}$	—
			256	$\mathcal{S}_{lr}$	—
		Lorsa	64	$\mathcal{S}_{lr}$	—
			128	$\mathcal{S}_{lr}$	—
			256	$\mathcal{S}_{lr}$	—
Qwen3 8B	$8\times$	Transcoder	256	1×10^{-4}	—
		Lorsa	256	9×10^{-5}	—
	$32\times$	Transcoder	256	8×10^{-5}	—
		Lorsa	256	6×10^{-5}	—

Table 5: Training hyperparameters for Gemma 3 JumpReLU Lorsa modules. Each model family uses separate small- and big- L_0 runs. Dashes are placeholders for the final released hyperparameters.

Model	Run	Width	Target L_0	Learning rate	Token batch size	λ	λ_{aux}	λ_{aux} warmup	Threshold LR factor	Threshold-freeze L_0	Training tokens
Gemma 3 270M	Small	–	–	–	–	–	–	–	–	–	–
	Big	–	–	–	–	–	–	–	–	–	–
Gemma 3 1B	Small	–	–	–	–	–	–	–	–	–	–
	Big	–	–	–	–	–	–	–	–	–	–
Gemma 3 4B	Small	–	–	–	–	–	–	–	–	–	–
	Big	–	–	–	–	–	–	–	–	–	–